

RIPHAH INTERNATIONAL UNIVERSITY

Faculty of Computing
Department of Computer Science

Car Rental Management System

Database Management System - Semester Project Report

Student Name	Talha Aftab
SAP ID	63091
Section	BSCS 4-1
Instructor	Mr. Ihtisham Ullah
Course	Database Management System
Semester	4th
Session	Spring 2026
University	Riphah International University Islamabad I-14
Submission Date	4 May 2026

1. Introduction

The Car Rental Management System is a relational database project designed to manage the core operations of a vehicle rental business. The system stores and organizes information related to customers, vehicles, bookings, payments, and staff, ensuring data consistency and reducing the manual workload associated with rental operations.

The project applies the database concepts covered throughout the semester, including entity-relationship modeling, relational schema design, normalization up to the Third Normal Form (3NF), constraint enforcement, and advanced SQL query writing. The implementation is carried out using Oracle SQL, which was used in lab sessions throughout the course.

The system is capable of tracking vehicle availability in real time, managing booking schedules, calculating rental costs automatically, and generating revenue and usage reports through SQL queries and views. These features make it a practical and complete database solution for a small to medium-scale car rental operation.

2. Objectives

The following objectives were defined for this project:

- Design a structured and normalized relational database for a car rental business.
- Model all entities, attributes, and relationships using the Chen notation Entity-Relationship Diagram.
- Implement the database schema in Oracle SQL with appropriate primary keys, foreign keys, and constraints.
- Normalize the schema to Third Normal Form (3NF) to eliminate data redundancy.
- Automate derived values such as rental duration and total cost using Oracle triggers.
- Create meaningful database views for reporting purposes.
- Write advanced SQL queries covering joins, subqueries, and aggregation functions.
- Populate the database with realistic sample data to demonstrate query functionality.

3. Entity-Relationship Diagram

The Entity-Relationship (ER) diagram below follows Chen notation as taught in the course. Entities are represented by rectangles, attributes by ovals, primary key attributes by underlined ovals, derived attributes by dashed ovals, and relationships by diamonds. Cardinality ratios are marked on the connecting lines.

3.1 Entities and Attributes

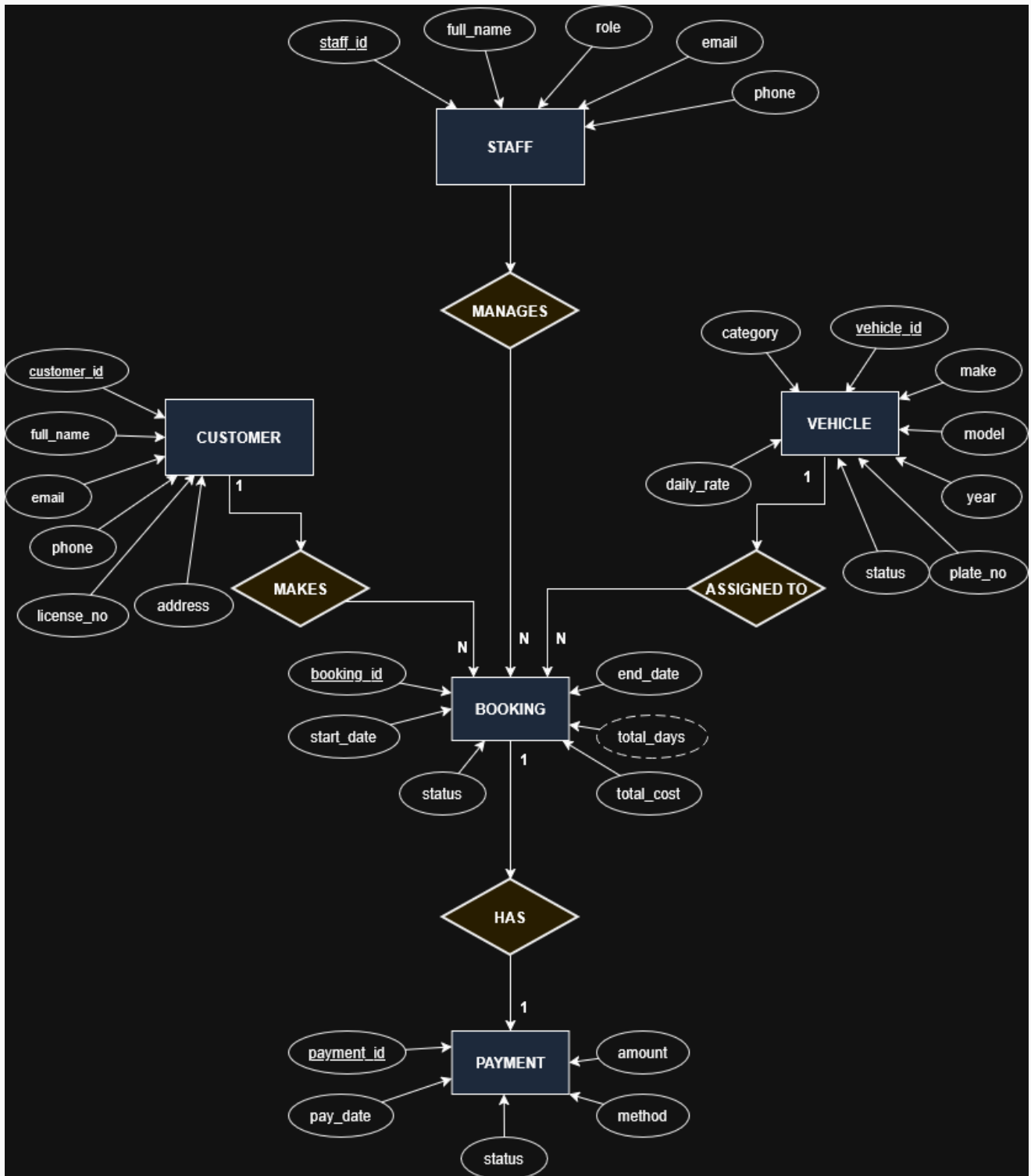
Entity	Attributes	Primary Key
CUSTOMER	customer_id, full_name, email, phone, license_no, address, created_at	customer_id
VEHICLE	vehicle_id, make, model, year, plate_no, category, daily_rate, status	vehicle_id
STAFF	staff_id, full_name, role, email, phone	staff_id
BOOKING	booking_id, customer_id (FK), vehicle_id (FK), staff_id (FK), start_date, end_date, total_days*, total_cost, status	booking_id
PAYMENT	payment_id, booking_id (FK), pay_date, amount, method, status	payment_id

* total_days is a derived attribute, computed from end_date minus start_date (shown as dashed oval in the ERD).

3.2 Relationships

Relationship	Entities Involved	Cardinality	Description
MAKES	CUSTOMER, BOOKING	1 : N	One customer can make many bookings
ASSIGNED TO	VEHICLE, BOOKING	1 : N	One vehicle can appear in many bookings
MANAGES	STAFF, BOOKING	1 : N	One staff member can manage many bookings
HAS	BOOKING, PAYMENT	1 : 1	Each booking has exactly one payment

3.3 ER Diagram



4. Schema Design

The relational schema was derived directly from the ER diagram. Each entity was translated into a table with appropriate columns, data types, primary keys, foreign keys, and constraints. Oracle SQL sequences and triggers were used to implement auto-incrementing primary keys, as the project runs on Oracle 11g Express Edition which does not support the IDENTITY syntax.

4.1 Normalization

All tables in the schema satisfy the requirements of the Third Normal Form (3NF).

First Normal Form (1NF): All tables contain only atomic values. Each column holds a single value, there are no repeating groups, and every row is uniquely identified by its primary key.

Second Normal Form (2NF): All tables have single-column primary keys, so partial dependencies cannot exist. Every non-key attribute depends on the entire primary key.

Third Normal Form (3NF): No transitive dependencies exist in any table. Non-key attributes depend only on the primary key and not on other non-key attributes. For example, in the BOOKING table, total_cost depends on the vehicle daily rate and the rental duration, both of which are resolved through the foreign key to VEHICLE and the computed trigger logic, not through another non-key attribute.

4.2 Tables

Table 1: STAFF

Column	Data Type	Constraint	Description
staff_id	NUMBER	PRIMARY KEY	Auto-generated unique identifier
full_name	VARCHAR2(100)	NOT NULL	Full name of the staff member
role	VARCHAR2(50)	NOT NULL, CHECK (Manager/Agent/Mechanic)	Staff role
email	VARCHAR2(100)	NOT NULL, UNIQUE	Staff email address
phone	VARCHAR2(20)	NOT NULL	Contact phone number

Table 2: CUSTOMER

Column	Data Type	Constraint	Description
customer_id	NUMBER	PRIMARY KEY	Auto-generated unique identifier
full_name	VARCHAR2(100)	NOT NULL	Full name of the customer
email	VARCHAR2(100)	NOT NULL, UNIQUE	Customer email address

phone	VARCHAR2(20)	NOT NULL	Contact number
license_no	VARCHAR2(50)	NOT NULL, UNIQUE	Driving license number
address	VARCHAR2(255)	NULL	Residential address
created_at	DATE	DEFAULT SYSDATE	Account creation date

Table 3: VEHICLE

Column	Data Type	Constraint	Description
vehicle_id	NUMBER	PRIMARY KEY	Auto-generated unique identifier
make	VARCHAR2(50)	NOT NULL	Vehicle manufacturer
model	VARCHAR2(50)	NOT NULL	Vehicle model name
year	NUMBER(4)	CHECK (2000-2030)	Manufacturing year
plate_no	VARCHAR2(20)	NOT NULL, UNIQUE	Registration plate number
category	VARCHAR2(30)	CHECK (Economy/Standard/SUV/Luxury/Van)	Vehicle category
daily_rate	NUMBER(8,2)	CHECK > 0	Daily rental rate in PKR
status	VARCHAR2(20)	DEFAULT Available	Current availability status

Table 4: BOOKING

Column	Data Type	Constraint	Description
booking_id	NUMBER	PRIMARY KEY	Auto-generated unique identifier
customer_id	NUMBER	FOREIGN KEY -> CUSTOMER	References customer
vehicle_id	NUMBER	FOREIGN KEY -> VEHICLE	References vehicle
staff_id	NUMBER	FOREIGN KEY -> STAFF	References staff
start_date	DATE	NOT NULL	Rental start date
end_date	DATE	NOT NULL, > start_date	Rental end date
total_days	NUMBER	Computed by trigger	Duration in days (derived)
total_cost	NUMBER(10,2)	Computed by trigger	Total rental cost in PKR
status	VARCHAR2(20)	CHECK (Pending/Active/Completed/Cancelled)	Booking status

Table 5: PAYMENT

Column	Data Type	Constraint	Description
--------	-----------	------------	-------------

payment_id	NUMBER	PRIMARY KEY	Auto-generated unique identifier
booking_id	NUMBER	FOREIGN KEY -> BOOKING, UNIQUE	References booking (1:1)
pay_date	DATE	DEFAULT SYSDATE	Date of payment
amount	NUMBER(10,2)	CHECK > 0	Amount paid in PKR
method	VARCHAR2(30)	CHECK (Cash/Credit Card/Debit Card/Online)	Payment method
status	VARCHAR2(20)	DEFAULT Pending	Payment status

4.3 Schema SQL Script

The following script creates all tables, sequences, triggers, and views.

```
-- STAFF table and sequence
CREATE TABLE STAFF (
    staff_id    NUMBER          PRIMARY KEY,
    full_name   VARCHAR2(100)   NOT NULL,
    role        VARCHAR2(50)    NOT NULL
    CONSTRAINT chk_staff_role
        CHECK (role IN ('Manager', 'Agent', 'Mechanic')),
    email       VARCHAR2(100)   NOT NULL CONSTRAINT uq_staff_email UNIQUE,
    phone       VARCHAR2(20)    NOT NULL
);

CREATE SEQUENCE seq_staff_id START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER trg_staff_id
BEFORE INSERT ON STAFF FOR EACH ROW
BEGIN
    SELECT seq_staff_id.NEXTVAL INTO :NEW.staff_id FROM DUAL;
END;
/
```

(Remaining tables follow the same pattern. Full script provided in the appendix and GitHub repository.)

4.4 Views

Three views were created to simplify common reporting queries.

View Name	Description
vw_active_rentals	Shows all currently active bookings with customer and vehicle details
vw_revenue_by_category	Aggregates total revenue, booking count, and average revenue per vehicle category
vw_customer_history	Summarizes each customer's total rentals, total spending, and last rental date

5. SQL Queries and Outputs

This section presents all SQL queries written for the system, organized by category. Each query is accompanied by a brief statement of purpose and the corresponding output captured from Oracle SQL Developer.

5.1 Join Queries

Query 1: All Bookings with Customer, Vehicle and Staff Details

Purpose: Retrieves full details of every booking by joining all four core tables.

```
SELECT b.booking_id, c.full_name AS customer_name,
       v.make || ' ' || v.model AS vehicle,
       v.plate no, s.full name AS handled by,
       b.start_date, b.end_date, b.total_days,
       b.total cost, b.status
FROM   BOOKING b
JOIN   CUSTOMER c ON b.customer_id = c.customer_id
JOIN   VEHICLE v ON b.vehicle_id = v.vehicle_id
JOIN   STAFF s ON b.staff_id = s.staff_id
ORDER BY b.booking_id;
```

Output:

BOOKING_ID	CUSTOMER_NAME	VEHICLE	PLATE_NO	HANDLED_BY	START_DATE	END_DATE	TOTAL_DAYS	TOTAL_COST	STATUS
1	Talha Aftab	Toyota Corolla	LHR-1234	Sara Khan	04/01/2026	04/04/2026	3	15000	Completed
2	Hamza Tariq	Honda Civic	ISB-5678	Bilal Hussain	04/05/2026	04/08/2026	3	16500	Completed
3	Zara Ahmed	Toyota Fortuner	RWP-3456	Sara Khan	04/10/2026	04/15/2026	5	60000	Completed
4	Ali Hassan	Toyota Camry	ISB-2345	Ahmed Raza	04/12/2026	04/14/2026	2	30000	Completed
5	Ayesha Noor	Suzuki Wagon R	RWP-0123	Fatima Malik	04/18/2026	04/21/2026	3	9000	Completed
6	Omar Farooq	Toyota Corolla	LHR-1234	Bilal Hussain	04/22/2026	04/25/2026	3	15000	Completed
7	Sana Ijaz	Suzuki Alto	KHI-9012	Sara Khan	04/25/2026	04/27/2026	2	5000	Completed
8	Kamran Bashir	Honda BRV	LHR-7890	Fatima Malik	04/28/2026	05/02/2026	4	32000	Completed
9	Talha Aftab	Toyota Camry	ISB-2345	Ahmed Raza	05/01/2026	05/04/2026	3	45000	Active
10	Zara Ahmed	Honda Civic	ISB-5678	Bilal Hussain	05/02/2026	05/06/2026	4	22000	Active

More than 10 rows available. Increase rows selector to view more rows.

Query 2: Completed Bookings with Payment Details

Purpose: Lists all completed bookings joined with their corresponding payment records.

```
SELECT b.booking_id, c.full_name AS customer_name,
       v.make || ' ' || v.model AS vehicle,
       b.total cost, p.amount AS amount_paid,
       p.method AS payment_method,
       p.pay_date, p.status AS payment_status
FROM   BOOKING b
JOIN   CUSTOMER c ON b.customer_id = c.customer_id
JOIN   VEHICLE v ON b.vehicle_id = v.vehicle_id
JOIN   PAYMENT p ON b.booking_id = p.booking_id
WHERE  b.status = 'Completed';
```

```

JOIN    PAYMENT p ON b.booking_id = p.booking_id
WHERE   b.status = 'Completed'
ORDER  BY p.pay_date;

```

Output:

BOOKING_ID	CUSTOMER_NAME	VEHICLE	TOTAL_COST	AMOUNT_PAID	PAYMENT_METHOD	PAY_DATE	PAYMENT_STATUS
1	Talha Aftab	Toyota Corolla	15000	15000	Cash	04/04/2026	Completed
2	Hamza Tariq	Honda Civic	16500	16500	Credit Card	04/08/2026	Completed
4	Ali Hassan	Toyota Camry	30000	30000	Debit Card	04/14/2026	Completed
3	Zara Ahmed	Toyota Fortuner	60000	60000	Online	04/15/2026	Completed
5	Ayesha Noor	Suzuki Wagon R	9000	9000	Cash	04/21/2026	Completed
6	Omar Farooq	Toyota Corolla	15000	15000	Online	04/25/2026	Completed
7	Sana Ijaz	Suzuki Alto	5000	5000	Cash	04/27/2026	Completed
8	Kamran Bashir	Honda BRV	32000	32000	Credit Card	05/02/2026	Completed

Query 3: Customer Booking Count (Including Customers with No Bookings)

Purpose: Uses a LEFT JOIN to include customers who have not made any bookings.

```

SELECT c.customer_id, c.full_name, c.phone,
       COUNT(b.booking_id) AS total_bookings
FROM   CUSTOMER c
LEFT JOIN BOOKING b ON c.customer_id = b.customer_id
GROUP BY c.customer_id, c.full_name, c.phone
ORDER BY total_bookings DESC;

```

Output:

CUSTOMER_ID	FULL_NAME	PHONE	TOTAL_BOOKINGS
1	Talha Aftab	0311-1111111	2
3	Zara Ahmed	0313-3333333	2
2	Hamza Tariq	0312-2222222	2
5	Ayesha Noor	0315-5555555	2
6	Omar Farooq	0316-6666666	1
8	Kamran Bashir	0318-8888888	1
7	Sana Ijaz	0317-7777777	1
4	Ali Hassan	0314-4444444	1

Query 4: Active Rentals with Customer Contact and Vehicle Details

Purpose: Shows all currently ongoing rentals with customer contact information.

```

SELECT b.booking_id, c.full_name AS customer_name,
       c.phone AS customer_phone,

```

```

        v.make || ' ' || v.model AS vehicle,
        v.plate_no, v.category,
        b.start_date, b.end_date,
        b.total_days, b.total_cost
FROM   BOOKING b
JOIN    CUSTOMER c ON b.customer_id = c.customer_id
JOIN    VEHICLE v ON b.vehicle_id = v.vehicle_id
WHERE  b.status = 'Active'
ORDER  BY b.start_date;

```

Output:

BOOKING_ID	CUSTOMER_NAME	CUSTOMER_PHONE	VEHICLE	PLATE_NO	CATEGORY	START_DATE	END_DATE	TOTAL_DAYS	TOTAL_COST
9	Talha Aftab	0311-1111111	Toyota Camry	ISB-2345	Luxury	05/01/2026	05/04/2026	3	45000
10	Zara Ahmed	0313-3333333	Honda Civic	ISB-5678	Standard	05/02/2026	05/06/2026	4	22000
11	Ayesha Noor	0315-5555555	Toyota Fortuner	RWP-3456	SUV	05/03/2026	05/07/2026	4	48000

Query 5: Revenue Generated per Staff Member

Purpose: Calculates the total revenue and number of bookings handled by each staff member.

```

SELECT s.staff_id, s.full_name AS staff_name, s.role,
       COUNT(b.booking_id) AS bookings_handled,
       SUM(p.amount) AS total_revenue_generated
FROM   STAFF s
JOIN    BOOKING b ON s.staff_id = b.staff_id
JOIN    PAYMENT p ON b.booking_id = p.booking_id
WHERE  p.status = 'Completed'
GROUP  BY s.staff_id, s.full_name, s.role
ORDER  BY total_revenue_generated DESC;

```

Output:

STAFF_ID	STAFF_NAME	ROLE	BOOKINGS_HANDLED	TOTAL_REVENUE_GENERATED
2	Sara Khan	Agent	3	80000
4	Fatima Malik	Agent	2	41000
3	Bilal Hussain	Agent	2	31500
1	Ahmed Raza	Manager	1	30000

5.2 Subquery Queries**Query 6: Customers Who Spent More Than the Average Customer Spending**

Purpose: Uses a nested subquery to compare each customer's total spending against the overall average.

```

SELECT c.customer_id, c.full_name, c.email,
       SUM(b.total_cost) AS total_spent
FROM   CUSTOMER c

```

```

JOIN   BOOKING  b ON c.customer_id = b.customer_id
GROUP BY c.customer_id, c.full_name, c.email
HAVING SUM(b.total_cost) > (
    SELECT AVG(total_per_customer)
    FROM (
        SELECT SUM(total_cost) AS total_per_customer
        FROM   BOOKING
        GROUP BY customer_id
    )
)
ORDER BY total_spent DESC;

```

Output:

CUSTOMER_ID	FULL_NAME	EMAIL	TOTAL_SPENT
3	Zara Ahmed	zara.ahmed@gmail.com	82000
1	Talha Aftab	talha.aftab@gmail.com	60000
5	Ayesha Noor	ayesha.noor@gmail.com	57000

Query 7: Vehicles That Have Never Been Booked

Purpose: Uses a NOT IN subquery to identify vehicles with no booking records.

```

SELECT vehicle_id,
       make || ' ' || model AS vehicle,
       plate_no, category, daily_rate, status
FROM   VEHICLE
WHERE  vehicle_id NOT IN (
    SELECT DISTINCT vehicle_id FROM BOOKING
);

```

Output:

VEHICLE_ID	VEHICLE	PLATE_NO	CATEGORY	DAILY_RATE	STATUS
10	Mercedes C-Class	ISB-8901	Luxury	25000	Available
7	Hyundai Tucson	KHI-6789	SUV	10000	Available
9	Toyota HiAce	LHR-4567	Van	18000	Maintenance

Query 8: Most Expensive Booking with Full Details

Purpose: Uses a scalar subquery to locate the booking with the highest total cost.

```

SELECT b.booking_id, c.full_name AS customer_name,
       v.make || ' ' || v.model AS vehicle,
       b.total_days, b.total_cost
FROM   BOOKING  b
JOIN   CUSTOMER c ON b.customer_id = c.customer_id
JOIN   VEHICLE  v ON b.vehicle_id = v.vehicle_id

```

```
WHERE b.total cost = (
    SELECT MAX(total_cost) FROM BOOKING
);
```

Output:

BOOKING_ID	CUSTOMER_NAME	VEHICLE	TOTAL_DAYS	TOTAL_COST
3	Zara Ahmed	Toyota Fortuner	5	60000

Query 9: Customers Who Have Rented an SUV

Purpose: Uses an IN subquery to find customers with at least one SUV booking.

```
SELECT DISTINCT c.customer_id, c.full_name,
               c.phone, c.email
FROM   CUSTOMER c
WHERE  c.customer_id IN (
    SELECT b.customer_id
    FROM   BOOKING b
    JOIN   VEHICLE v ON b.vehicle_id = v.vehicle_id
    WHERE  v.category = 'SUV'
);
```

Output:

CUSTOMER_ID	FULL_NAME	PHONE	EMAIL
3	Zara Ahmed	0313-3333333	zara.ahmed@gmail.com
5	Ayesha Noor	0315-5555555	ayesha.noor@gmail.com
8	Kamran Bashir	0318-8888888	kamran.bashir@gmail.com

Query 10: Vehicles with Daily Rate Above Average

Purpose: Uses a scalar subquery to filter vehicles priced above the fleet average.

```
SELECT vehicle id,
       make || ' ' || model AS vehicle,
       category, daily_rate, status
FROM   VEHICLE
WHERE  daily_rate > (
    SELECT AVG(daily_rate) FROM VEHICLE
)
ORDER BY daily_rate DESC;
```

Output:

VEHICLE_ID	VEHICLE	CATEGORY	DAILY_RATE	STATUS
10	Mercedes C-Class	Luxury	25000	Available
9	Toyota HiAce	Van	18000	Maintenance
6	Toyota Camry	Luxury	15000	Rented
4	Toyota Fortuner	SUV	12000	Rented

5.3 Aggregation Queries

Query 11: Overall Revenue and Booking Statistics

Purpose: Provides a summary of total revenue, average booking value, and payment range.

```
SELECT COUNT(b.booking_id)      AS total_bookings,
       SUM(p.amount)           AS total_revenue,
       ROUND(AVG(p.amount), 2) AS avg_booking_value,
       MAX(p.amount)           AS highest_payment,
       MIN(p.amount)           AS lowest_payment
FROM   BOOKING b
JOIN   PAYMENT p ON b.booking_id = p.booking_id
WHERE  p.status = 'Completed';
```

Output:

TOTAL_BOOKINGS	TOTAL_REVENUE	AVG_BOOKING_VALUE	HIGHEST_PAYMENT	LOWEST_PAYMENT
8	182500	22812.5	60000	5000

Query 12: Revenue Breakdown by Vehicle Category

Purpose: Groups completed payment data by vehicle category to show which category generates the most revenue.

```
SELECT v.category,
       COUNT(b.booking_id)      AS total_bookings,
       SUM(p.amount)           AS total_revenue,
       ROUND(AVG(b.total_days), 1) AS avg_rental_days,
       MAX(p.amount)           AS max_payment
FROM   VEHICLE v
JOIN   BOOKING b ON v.vehicle_id = b.vehicle_id
JOIN   PAYMENT p ON b.booking_id = p.booking_id
WHERE  p.status = 'Completed'
GROUP BY v.category
ORDER BY total_revenue DESC;
```

Output:

CATEGORY	TOTAL_BOOKINGS	TOTAL_REVENUE	AVG_RENTAL_DAYS	MAX_PAYMENT
SUV	2	92000	4.5	60000
Standard	3	46500	3	16500
Luxury	1	30000	2	30000
Economy	2	14000	2.5	9000

Query 13: Monthly Booking and Revenue Summary

Purpose: Uses TO_CHAR to group bookings by month and summarizes revenue collected each month.

```
SELECT TO_CHAR(b.start_date, 'YYYY-MM') AS month,
       COUNT(b.booking_id)              AS total_bookings,
       SUM(b.total_cost)                 AS total_cost,
       SUM(p.amount)                    AS collected_revenue
FROM   BOOKING b
JOIN   PAYMENT p ON b.booking_id = p.booking_id
WHERE  p.status = 'Completed'
GROUP BY TO_CHAR(b.start_date, 'YYYY-MM')
ORDER BY month;
```

Output:

MONTH	TOTAL_BOOKINGS	TOTAL_COST	COLLECTED_REVENUE
2026-04	8	182500	182500

Query 14: Payment Method Usage Breakdown

Purpose: Counts transactions and totals revenue per payment method to identify the most used methods.

```
SELECT method,
       COUNT(*)              AS total_transactions,
       SUM(amount)           AS total_amount,
       ROUND(AVG(amount), 2) AS avg_amount
FROM   PAYMENT
WHERE  status = 'Completed'
GROUP BY method
ORDER BY total_amount DESC;
```

Output:

METHOD	TOTAL_TRANSACTIONS	TOTAL_AMOUNT	AVG_AMOUNT
Online	2	75000	37500
Credit Card	2	48500	24250
Debit Card	1	30000	30000
Cash	3	29000	9666.67

Query 15: Top 3 Most Rented Vehicles

Purpose: Uses ROWNUM with a subquery to return the three vehicles with the most bookings.

```
SELECT * FROM (
  SELECT v.vehicle_id,
         v.make || '-' || v.model AS vehicle,
         v.plate_no, v.category,
         COUNT(b.booking_id) AS times_rented
  FROM   VEHICLE v
  JOIN   BOOKING b ON v.vehicle_id = b.vehicle_id
  GROUP BY v.vehicle_id, v.make, v.model,
           v.plate_no, v.category
  ORDER BY times_rented DESC
)
WHERE ROWNUM <= 3;
```

Output:

VEHICLE_ID	VEHICLE	PLATE_NO	CATEGORY	TIMES_RENTED
2	Honda Civic	ISB-5678	Standard	2
4	Toyota Fortuner	RWP-3456	SUV	2
6	Toyota Camry	ISB-2345	Luxury	2

5.4 View Queries

Query 16: Active Rentals View

Purpose: Queries the vw_active_rentals view to display all ongoing rentals.

```
SELECT * FROM vw_active_rentals;
```

Output:

BOOKING_ID	CUSTOMER_NAME	CUSTOMER_PHONE	VEHICLE	PLATE_NO	START_DATE	END_DATE	TOTAL_DAYS	TOTAL_COST
9	Talha Aftab	0311-1111111	Toyota Camry	ISB-2345	05/01/2026	05/04/2026	3	45000
10	Zara Ahmed	0313-3333333	Honda Civic	ISB-5678	05/02/2026	05/06/2026	4	22000
11	Ayesha Noor	0315-5555555	Toyota Fortuner	RWP-3456	05/03/2026	05/07/2026	4	48000

Query 17: Revenue by Category View

Purpose: Queries the vw_revenue_by_category view to show revenue grouped by vehicle type.

```
SELECT * FROM vw_revenue_by_category;
```

Output:

CATEGORY	TOTAL_BOOKINGS	TOTAL_REVENUE	AVG_REVENUE
Economy	2	14000	7000
Luxury	1	30000	30000
Standard	3	46500	15500
SUV	2	92000	46000

Query 18: Customer History View

Purpose: Queries the vw_customer_history view to show each customer's rental history and total spending.

```
SELECT * FROM vw_customer_history ORDER BY total_spent DESC;
```

Output:

CUSTOMER_ID	FULL_NAME	TOTAL_RENTALS	TOTAL_SPENT	LAST_RENTAL_DATE
3	Zara Ahmed	2	82000	05/06/2026
1	Talha Aftab	2	60000	05/04/2026
5	Ayesha Noor	2	57000	05/07/2026
8	Kamran Bashir	1	32000	05/02/2026
4	Ali Hassan	1	30000	04/14/2026
2	Hamza Tariq	2	22500	04/08/2026
6	Omar Farooq	1	15000	04/25/2026
7	Sana Ijaz	1	5000	04/27/2026

6. Conclusion

This project demonstrates the end-to-end design and implementation of a relational database system for a car rental business using Oracle SQL. Beginning with a conceptual Entity-Relationship Diagram and progressing through schema design, normalization, and query development, the project covers all major topics studied in the Database Management System course.

The schema comprises five interrelated tables, each enforcing data integrity through primary keys, foreign keys, check constraints, and unique constraints. Triggers were used to automate the computation of rental duration and cost, reducing the possibility of data entry errors. Three views were created to simplify access to commonly needed reports.

Eighteen SQL queries were written to demonstrate proficiency in inner joins, left joins, multi-table joins, correlated subqueries, nested subqueries, and aggregation functions including COUNT, SUM, AVG, MAX, and MIN. The queries cover practical reporting scenarios such as revenue analysis, vehicle utilization, customer spending patterns, and payment tracking.

Overall, the project provided hands-on experience with the complete database development lifecycle and reinforced the theoretical concepts covered throughout the semester.

7. GitHub Repository

The complete project including the schema script, data insertion script, and all SQL queries has been uploaded to the following GitHub repository:

GitHub Repository Link: [Github Link](#)

Repository contents:

- `car_rental_schema_11g.sql` - Full schema with tables, sequences, triggers, and views
- `car_rental_data.sql` - Sample data insertion script
- `car_rental_queries.sql` - All 18 SQL queries
- `README.md` - Project description and setup instructions